The Aztec Market System

How a marketplace works when almost everything about it is a secret

Written for readers with no prior knowledge of Aztec or zero-knowledge cryptography.

What this document describes

A marketplace framework built on the Aztec network in which markets are invisible unless you hold their link, a vendor's real identity never appears anywhere, and a buyer can pay for an item while the blockchain itself verifies the payment was correct, *without the amount, the item, the buyer, or the seller ever becoming public*. This document explains each mechanism, and is candid about what each one does and does not hide.

Contents

1. Aztec in five minutes

Most blockchains are radically public. On Ethereum, every account balance, every payment, and every interaction with an application is visible to anyone, forever. **Aztec** is a network built on top of Ethereum that takes the opposite stance: by default what you do is your own business, and the chain verifies your actions without learning them.

Four ideas are enough to follow everything that comes later.

Public state and private state

An Aztec contract keeps two kinds of data. **Public state** behaves like ordinary blockchain storage: a value at a slot, readable by anyone. **Private state** is stored as **notes**: encrypted records that live in your own device's database. The chain never sees a note's contents. It sees only a *commitment*, a hash that proves the note exists without revealing what is inside it.

Spending a note means publishing a **nullifier**: a one-way tag derived from the note. The network keeps a list of every nullifier it has ever seen and rejects duplicates. That single rule prevents double-spending without anyone learning which note was spent, by whom, or for how much. A nullifier is an opaque number and cannot be traced back to its note.

Private functions run on your own machine

Here is the counterintuitive part. When you call a private function, it does not execute on the network. It executes locally, on your computer, and produces a **zero-knowledge proof** that it executed correctly. You send the proof, not the inputs. Validators check the proof and learn nothing else.

This is what makes a private marketplace possible at all. The contract can insist that the amount you escrowed matches the listing's price, and be certain of it, while the price and the amount stay secret. The rule is enforced by mathematics rather than by someone being trusted to look.

> **The one-sentence version**
> On Aztec, “the contract verified my payment” does not mean a computer somewhere looked at your payment. It means *your own machine* ran the contract's rules over your secret data and produced a proof that the rules held, and the network only ever sees the proof.

One consequence shapes much of the design: a private function cannot read public state at the moment it runs, because it runs before the transaction is ordered. Instead it reads a value from a recent block and proves the value was there, using the block header as an anchor. Wherever this document says a private function “proves” something about public storage, that is the mechanism.

Encrypted messages ride along

If a note belongs to you, you need to receive it. Aztec transactions carry encrypted logs addressed to a recipient's key. Your wallet scans them, decrypts what it can open, and stores the notes it finds. This is how a vendor learns they have an order, and how a buyer receives a receipt, with no server in the middle.

Fees

Every transaction costs fee juice, and fee juice has to come from an account, which is awkward when the entire point is that nobody should know who you are. Section 12 explains how the system pays for transactions without tying them to an identity.

2. The pieces of the system

The system is a framework: anyone can deploy and operate their own independent market. The parts are one contract per market, a shared library, a global lookup table, and a desktop application.

Piece	Role
Marketplace (contract)	One instance per market. Holds the configuration, vendor records, listings, orders, feedback, and moderation state, plus vendor deposits. It never holds order funds.
OrderEscrow (contract)	One instance per order, holding that order's funds. Nobody deploys it and its class is never published: it exists because both parties can derive its address. It pays out only against a terminal order state the Marketplace has already recorded.
MarketplaceRegistry (contract)	One global lookup table shared by every market. Entries are keyed by an opaque fingerprint of a market's access secret, so it resolves a link you already hold rather than advertising anything.
market-protocol (library)	Shared domain constants and Poseidon2 derivations, mirrored exactly in TypeScript and cross-checked with identical test vectors, so the app and the circuit can never disagree about a hash.
cUSDC (external contract)	The currency. An already-deployed token on Aztec with private balances, held at a fixed address per network. Nothing in this system deploys it; markets simply point at it.
Desktop app	Everything a person does: browsing, buying, selling, moderating, messaging, and deploying a market.
Arweave	Permanent storage for encrypted listing documents. It stores ciphertext and never sees a key.

The contracts are written in **Noir**, Aztec's language for zero-knowledge programs; everything else is TypeScript. There is no server, no indexer, and no API of our own. The app talks to an Aztec node and to Arweave, and derives everything else itself.

The currency

Trade is denominated in **cUSDC**, a dollar-denominated token that already exists on Aztec at a known address on each network and supports private balances. This system does not issue, deploy, or govern it. Users acquire cUSDC by bridging, and a market simply records its address as the asset orders are priced in.

This matters for privacy in a way that is easy to miss. Because the currency is shared with everyone else using it on Aztec, market activity does not stand out as belonging to a particular marketplace's own coin. A bespoke token would be a label on every transaction that touched it.

A market records the payment asset as an address, so it is technically free to point elsewhere, and the local development sandbox does exactly that: it deploys a mock cUSDC, because no real one exists there. In practice every real market uses the network's cUSDC, which is what the deployment wizard fills in.

3. Two wallets, one seed

Every user has a single recovery seed, and that seed produces two very different kinds of account. Understanding the split makes the rest of the system fall into place.

The **universal wallet** is the account that touches the outside world. It holds your tokens, it holds the credit that pays transaction fees, and it is the account you bridge funds into from Ethereum. It is the closest thing here to a conventional wallet.

A **per-market account** is derived from your seed combined with a specific market's link and an index. It is the identity that actually appears on-chain inside that market: the vendor who owns a listing, the moderator who approves one, the owner who deployed it. Because the market's link is folded into the derivation, the same person selling on two markets produces two unrelated accounts, and neither reveals anything about the universal wallet behind them.

You obtain a per-market account by claiming a username. The contract stores only `hash(username, access secret)` alongside the address that claimed it, so the plaintext handle never reaches the chain. Anyone holding the market link can still verify that a listing's displayed handle genuinely belongs to its creator, by recomputing that hash and comparing. Owners and vendors may hold several identities in one market and switch between them in the app.

> **The important consequence**
> A per-market account is an *inbox and a name*. It never holds fee juice and never needs funding before it can act. Money that arrives for it can be swept to the universal wallet whenever its owner chooses. This is what stops a pseudonym from being linked to the wallet that pays for it (see section 12).

4. Hidden markets: the market link

There is no directory of markets. Every market is hidden, the way an onion service is hidden: reachable only by someone who was given its address. That address is the **market link**, and what it really carries is an **access secret**, a random 254-bit number generated at deployment and shared out of band, in a chat message, a QR code, or a whisper. The resemblance to an onion service is deliberate right down to the format: a link is that secret rendered as a 56-character address ending in `.aztec`, and it contains nothing else, not even the market's contract address.

Finding a market you already know about

Every market registers itself in one global registry contract. That sounds like it should defeat the whole purpose, and it would if the registry were keyed by anything meaningful. It is keyed by a one-way fingerprint of the access secret instead:

`contracts/market-protocol/src/lib.nr`

```
pub fn derive_market_lookup_key(access_secret: Field) -> Field {
    poseidon2_hash([access_secret, DOMAIN_MARKET_LOOKUP])
}
```

Holding the secret, you recompute the key and find the market's contract address immediately. Without it, you are looking at rows keyed by meaningless numbers. Scanning the registry tells an outsider how

many markets exist and nothing whatever about any of them: not a name, not a category, not an owner.

Registration is permissionless and first-come-first-served per key, and the registry cannot check that a key was derived honestly. It does not need to. Someone who squats a key they cannot decrypt has only pointed a link at a contract whose sealed content will not open, and the app refuses to render a market it cannot decrypt. Squatting produces a failure, never a convincing forgery.

The access secret is the master key to everything readable about the market. From it the app derives:

- the registry lookup key, which turns the link into a contract address;
- the key that decrypts the storefront document (name, description, pages, categories, branding);
- the key that decrypts every listing document;
- the salt for category tags, so a category can be matched without its name being guessable;
- the blinding factors that let a reader check a listing's price against the commitment stored on-chain;
- nothing to do with money. It once derived the marketplace contract's own encryption keys; the contract is now deployed with none, and each order's escrow uses a secret minted for that order alone.

Each derivation is tagged with a distinct domain constant, so the same secret can feed several purposes with no risk that one derived value collides with or substitutes for another.

Someone without the link, looking directly at the contract, sees ciphertext, opaque hashes, and counters. They can tell that *a* market exists at that address and roughly how busy it is. They cannot tell what it sells, what anything costs, or even what it is called.

The encryption is authenticated, which the app exploits deliberately: decrypting with the wrong key does not yield garbage, it fails outright. A mistyped link or substituted ciphertext ends in “this market could not be opened” rather than in attacker-controlled content. Decryption *is* the authentication.

Sharing the link shares read access. There is no way to un-share it, and no way to read the market without it.

5. What the chain actually stores

It is worth being concrete about the split, because it determines exactly what leaks.

Public, on-chain	Private
Sealed storefront document (ciphertext)	Order contents, buyer identity, delivery details
Listing records: encrypted pointer, price commitment, category tag, status, creator, position	Listing document text, images, and every price in its table
Vendor records: address, status, deposit amount, active order count	Which listings a buyer viewed or bought
Market configuration: fee rate, deposit and collateral amounts, timeout, policies	Escrow balances, held as private notes at a separate address per order
Counters, nullifiers, and anonymous per-order flags	Feedback text and ratings, stored as sealed blobs

Notice what is public. The market's *rules* are public: the fee percentage, the deposit required of vendors, how long an order may sit before it settles. That is deliberate: those are the terms everyone is agreeing to, and they ought to be checkable. What stays private is every particular: who, what, and how much.

Per-subject collections (a category's listings, a vendor's listings, a listing's feedback) all live in a single generic map keyed by a domain-separated tag. Adding a new kind of collection means choosing a new tag rather than adding new storage, which keeps the contract's storage layout small enough for the compiler to handle.

6. Vendors without identities

A vendor's on-chain identity is simply their per-market account address. There is no registration document, no real name, and no link to anything outside the market.

Becoming a vendor takes two steps: claim a username, then register as a vendor from that same account. What happens next depends on the market's **vendor policy**, fixed by the owner at deployment:

Policy	Effect
Open	Anyone who registers becomes active immediately.
Approval	Registration stays pending until the owner or a moderator approves it.
Deposit	Registration requires locking a deposit in cUSDC.
Both	Deposit and approval.

A deposit is a stake against bad behaviour. It is paid *directly from the universal wallet* in the same transaction that registers the vendor, so the pseudonymous account never has to hold the funds and no transfer between the two ever appears. Recovering the deposit is deliberately slow: the vendor requests withdrawal, a delay elapses, and the deposit is returned only if they have no active orders. A vendor cannot take an order, take the money, and walk off with their stake the same afternoon.

Suspending a vendor hides their listings without touching their deposit or their outstanding orders, so moderation cannot be turned into a way to steal.

7. Listings: sealed content, verifiable prices

A listing has two halves: content that is encrypted and stored off-chain, and a small public record that commits to it.

The content (title, description, images, options, the vendor's handle) is validated, sealed under the access secret, and uploaded to Arweave, which returns a storage id. That id is itself a hash of the content, so it doubles as an integrity check: if the bytes change, the id no longer matches. The id is then encrypted under the access secret, and *that* is what goes on-chain, as a 61-byte pointer packed into two field elements.

So the chain holds an encrypted pointer to encrypted content. Without the market link you cannot even work out which Arweave object belongs to which listing.

Prices you can check but not see

The price is the interesting part. It must stay hidden, yet a buyer must not be able to pay less than it, and a vendor must not be able to change it after the fact.

The answer is a commitment. There is rarely just one price, though: a listing offers variants (sizes, colours) and shipping methods, each priced separately. So the commitment covers the whole **price table** at once: every variant price, then every shipping price, under a blinding factor derived from the access secret and the listing's id.

```
blinding   = poseidon2([access_secret, listing_id, DOMAIN_PRICE_BLINDING])
commitment = poseidon2([blinding, DOMAIN_PRICE_COMMITMENT,
                        variantCount, shippingCount,
                        variantPrices[0..7], shippingPrices[0..3]])
```

To an outsider the commitment is an unguessable hash. The blinding factor carries about 254 bits of entropy, so guessing plausible prices and checking them gets nowhere. To a link holder it is verifiable: recompute it from the decrypted document and confirm it matches. A listing whose displayed prices disagree with its on-chain commitment is refused by the app rather than shown.

And to a buyer it is something better still: a value they can *open inside a proof*. When an order is placed the buyer hands the circuit the whole table and says which row of each list they chose. The circuit rebuilds the commitment, checks it matches the chain, and then *computes the order total itself*:

```
amount = variantPrice[chosen] × quantity + shippingPrice[chosen]
```

Which is the point: the total is never something the buyer asserts and the contract believes. It is derived from prices the vendor committed to publicly, inside a proof, while the prices, the choice and the total all stay secret. Shipping is charged once per order; only the item scales with quantity.

A subtlety worth stating

The table is a fixed size (up to eight variants and four shipping methods), and unused rows are committed as *zeros*. That keeps the commitment a fixed shape and binds the row counts, so a vendor cannot quietly reveal an extra row later. But it also means an out-of-range choice would select a row priced at nothing. The circuit therefore rejects any index at or past the real count, and those two checks are the only thing standing between a listing and being bought for free.

Categories

A listing belongs to exactly one category, stored as a salted hash of the category name rather than the name itself. Link holders filter by category by computing the same tag; outsiders see an opaque number they cannot reverse, because the salt is the access secret.

Status and approval

Every listing carries a status: active, paused, removed, or pending approval. Markets configured to moderate listings hold new and edited listings as pending until a moderator approves them, and editing an approved listing returns it to pending, so content cannot be swapped in after review. Buyers only ever see active listings, and only active listings can be ordered.

8. The order of a category

The sequence in which listings appear inside a category is stored explicitly, as a linked list: the category records which listing comes first, and each listing records which one follows it.

New listings join the end. The owner, or a moderator holding the listings permission, can move any listing elsewhere: promoting it to the top of its category, or nudging it up and down a place at a time.

A move costs the same no matter how large the category has grown. The app walks the chain locally to find the listing's current neighbour and its intended one, and passes both to the contract, which verifies them before relinking. Searching the list inside the transaction would cost an unbounded number of storage reads; naming the neighbours and checking the claim costs a fixed few. A caller who lies about a neighbour has their transaction rejected rather than corrupting the order.

Because order is stored separately from identity, rearranging a listing changes nothing else about it: not its id, not its price commitment, not the content bound to it. Paused and removed listings keep their place in the sequence while staying invisible to buyers.

9. Fully private orders and escrow

This is the centrepiece, and the brief is demanding: the buyer pays into escrow, the contract verifies the amount matches the hidden price, the vendor receives the order and proof of payment, and an outside

observer sees nothing attributable at all.

Placing an order is a single private transaction. Inside it, and never leaving the buyer's machine, all of the following happen at once:

1. The listing's status and price commitment are proven against a recent block, so the item is genuinely active at the price claimed.
2. The listing's whole price table is opened against the commitment, and the total is computed from the rows the buyer chose, so the amount is derived rather than asserted.
3. Funds move from the buyer's private balance into that order's own escrow address: the item's price plus a small refundable collateral.
4. An order note is delivered to the vendor's inbox and a receipt note to the buyer, carrying which variant and shipping method were chosen.
5. A delivery memo, encrypted so that only the vendor can read it, is attached.

Two details deserve spelling out.

Where the money sits. Not in the marketplace. Each order's funds go to an address belonging to that order alone, holding encrypted notes exactly as a user would, its public balance never changing. The amount exists only inside sealed envelopes.

There is no pool, and that is the point. An earlier design did pool the funds in the marketplace contract under note keys derived from the market link. That worked, but it meant anyone holding the link could read the pool: every amount, every deposit, every payout. The key that let a buyer prove their settlement was the same key that revealed everyone else's orders. A shared pool cannot be private per order.

So each order gets its own escrow, with keys derived from a secret minted for that one order and shared with nobody but its two parties. A leaked secret exposes exactly one order. Sharing it with the vendor is deliberate: it is how they settle. And it cannot be used to steal, for a subtle reason. Holding a note's keys lets you build a proof *about* the note, but the token only moves when the escrow's own code is the caller, which happens exclusively inside the rules below.

Nobody deploys these escrows. Not the buyer, not the vendor, not the market owner, and no setup step deploys a template either. Both parties simply compute the address, and the network accepts calls to it, because an Aztec address is itself a commitment to the code that runs there and the terms it was opened with. This costs nothing and publishes nothing: an observer cannot even tell the address exists.

That commitment is also what proves the buyer paid. If buyers fund their own escrows, what stops one of them recording an order and funding nothing? The marketplace cannot tell by looking. An address is a hash of the code, the keys and the terms, and a proof has no way to compute one. So the marketplace does not check the escrow. It *opens* it. Placing an order works the price out from the listing's sealed price list, builds the escrow's terms from that price and the market's own settings, opens an escrow on exactly those terms at the address the buyer named, and only then moves the money in. The network permits that opening only if the address really is the one those terms produce. An escrow opened for a cheaper order, a friendlier vendor, a smaller deposit or a different market is a different address, and the order cannot be placed at all. So a vendor never has to check whether an order was paid for: an order that exists was paid for, in the same transaction that created it.

The catch is that a contract nobody deployed can have no public part, and a private-only contract cannot be ordered against a competing transaction. So the escrow never decides anything. The marketplace, which *can* settle a race in public, resolves every contested question and records the answer; the escrow reads that answer and pays. Which is why finishing an order takes two transactions: the marketplace first, then the money.

Who the buyer is. The buyer pays directly from their universal wallet, and the order note carries no buyer address at all. The vendor learns what was ordered and where to ship it, and nothing else. Neither the vendor nor the market operator can determine who bought what.

The order's identifier is derived from the market, the listing, and a random per-order number that the buyer keeps. It is opaque, it reveals nothing, and it is what both parties use to refer to the order afterwards.

The collateral is what makes feedback meaningful. It is the buyer's own money, returned when they confirm receipt and leave a rating, so a rating costs nothing to give and something to withhold.

10. Settlement, refunds, and timeouts

An order can end in exactly four ways.

Ending	Who acts	Result
Cancelled	Buyer, before the vendor accepts	The full escrow returns to the buyer.
Completed	Buyer, after receiving the goods	The vendor is paid, the market fee is split off, the collateral returns to the buyer, and their rating is recorded.
Refunded	Vendor, at any point	The full escrow, collateral included, is made available for the buyer to claim.
Timed out	Vendor, after the timeout	An accepted order the buyer never confirmed settles to the vendor.

Each of these is two transactions rather than one, and the reason is the interesting part. Every ending above is a race with some other ending: a buyer cancelling races the vendor accepting, and a vendor claiming a timeout races the buyer disputing. Deciding a race means being ordered against the other transaction, which only a public call can be, and the escrow, having no public part, cannot do it. So the marketplace decides. It runs the check in public, where the network puts transactions in a definite order, and records the verdict; if the vendor's acceptance landed first, the cancellation simply fails and the money stays put. Only then does the escrow act, and by that point there is nothing left to race: it reads a verdict that can never change and pays accordingly.

The practical consequence is that the money can lag the decision by a block, and if the second transaction is never sent the funds simply wait: the verdict is permanent, so the payout can always be claimed later.

Between placing and ending, the vendor accepts the order and can post shipping updates. Acceptance and fulfilment are recorded as anonymous public flags against the opaque order id, which is enough for both sides to follow progress, revealing nothing about either of them.

One ending, exactly once. Every settlement path publishes the same nullifier, derived from the order id. Because the network rejects duplicate nullifiers, an order reaches one terminal state one time. Double refunds, pay-then-refund races, and replayed confirmations are impossible by construction rather than by careful bookkeeping.

Cancel only before acceptance. That ordering needs a fact both sides agree on, and note-based state cannot provide one: you cannot prove a note does *not* exist yet. So acceptance writes a single public flag against the order id, and cancellation enqueues a public check that the flag is still unset. If the vendor's acceptance lands first, the cancellation reverts and the escrow is never authorised to pay out, so the money stays where it is. The race is settled by block ordering, and the leak is one anonymous bit.

Time without a clock. Private execution has no notion of “now”, since a proof can be generated at any moment. Block headers carry timestamps and only move forward, so a timeout claim proves against a recent header that the deadline had already passed: if a past block was late, now is later.

Where the money goes. When an order settles in the vendor's favour, the payment goes to the vendor's per-market account and the market's fee to the owner's. This is intentional. The vendor's pseudonym is what appears on the listing, so paying anywhere else would tie that pseudonym to another account. Both parties sweep their proceeds to their universal wallet whenever they like.

Refunds are claimed, not pushed. Because the contract does not know the buyer's address, it cannot send money to them. Instead it marks the escrow as theirs to take, and the buyer collects it in their own transaction. The alternative would have meant storing the buyer's address, precisely what the design exists to avoid.

11. Disputes

Arbitration happens in conversation, not in the contract. Buyer, vendor, and a moderator talk in the app's private messaging, built on the **SimpleX** protocol, whose privacy model mirrors the system's own: no user identifiers of any kind, relay queues that keep no records, and end-to-end encryption.

The chain's role is deliberately small. Marking an order disputed sets an anonymous public flag against the order id, and while it is set the vendor's timeout settlement will not go through. That single fact is what gives the conversation teeth: a disputed order can only end by the buyer confirming or the vendor refunding, so neither side can wait the other out.

The hard problem is proving to a moderator that you are the actual buyer, when nothing on-chain records who that is. The answer is a commitment published when the dispute opens. The buyer commits to a secret; later they reveal that secret in the dispute conversation; the moderator recomputes one hash and compares. The moderator learns that they are speaking to the genuine buyer, and nothing more: no address, no identity.

A moderator holding the disputes permission rules for one side or the other. The ruling is a flag, and the moderator never touches the money: it unlocks a payout that the winning party then collects from the order's own escrow, themselves. An arbiter who can decide but cannot pay themselves is a far smaller thing to trust.

Vendors and moderators publish a contact address so they can be reached: a SimpleX link, sealed under the access secret and written to the contract by the account it belongs to. Because only that account can write it, a successfully opened address is provably that pseudonym's: the on-chain write is the attestation.

Messaging identity follows account identity exactly. Each per-market account gets its own messaging profile, with its own conversations, contacts and display name. Two accounts on one market are two personas, which is the entire reason for having more than one, so they share nothing, and switching accounts switches inbox. The one deliberate exception is the market's dispute desk: its address is published on-chain and belongs to the market rather than to a person, so it stays put when the owner switches accounts, and every open dispute stays reachable.

12. Paying for transactions anonymously

Transactions cost fees, and a fee is paid by an account. Naively, a vendor accepting an order would pay from the vendor's account, which means funding that account, which means a transfer from somewhere, which is exactly the link the whole design is trying to avoid.

The system resolves this in two parts.

First, fees are paid through a **fee-paying contract** that accepts payment from a private balance. You bridge fee juice once into credit held inside that contract, and spend it privately thereafter. The contract is a shared, standard one rather than a bespoke deployment, so the anonymity set spans every application that uses it, not merely this marketplace.

Second, the universal wallet **acts on behalf of** the per-market account. Every market function takes the acting identity as an explicit argument and verifies it cryptographically. The universal wallet sends the transaction and pays the fee; the per-market account signs a one-time authorisation naming that exact call. On-chain the action is unambiguously the pseudonym's, while the gas came from an account nobody can connect to it.

The practical result is that pseudonymous accounts never need funding. A vendor can register, list, accept orders, and be paid without their pseudonym ever holding a fee.

13. Running a market

Deploying a market is done from the app. The owner chooses the presentation (name, description, pages, categories, branding) and the rules:

Setting	Meaning
Payment asset	The token orders are denominated in. Defaults to the network's cUSDC.
Fee	The market's cut of each completed order, in basis points.
Vendor policy	Open, approval, deposit, or both.
Vendor deposit	The stake required, where the policy demands one.
Order timeout	How long before an accepted order may be settled by the vendor. Also the delay on deposit withdrawal.
Finalisation collateral	The refundable amount a buyer escrows alongside the price.
Listing policy	Whether new and edited listings need approval.

The owner's username is registered as part of the deployment itself, so the market has a named owner from its first block without a second transaction.

The owner is also the market's treasury: fees accrue to the owner's per-market account. Ownership can be transferred to another account.

Owners appoint **moderators** by username and grant them a bitmap of permissions: moderate listings, manage vendors, resolve disputes. Moderators are pseudonymous like everyone else, and the owner implicitly holds every permission.

Presentation can be edited after deployment without redeploying. Changing the *rules* is a separate operation from changing the *storefront*, so a cosmetic edit cannot quietly alter the fee.

Finally, the app checks the contract it is talking to. Before opening a market it verifies the contract's class against a list of known-good builds, so a link pointing at a modified contract that merely looks like a marketplace is refused rather than trusted.

14. What an observer actually sees

The table below is the privacy contract, action by action. An “observer” is anyone watching the chain without the market link.

Event	Outside observer	Link holders	The parties
Market deployed	**A marketplace contract appeared, plus its config values, plus a registry row under a meaningless key, all unlabelled**	**Name, description, appearance, policies**	None
Listing created	**A listing exists; ciphertext size; an opaque price commitment and category tag**	**Title, description, price, vendor handle**	None
Vendor registered	**A pseudonym joined the roster; the deposit amount, already public in the config**	**same**	**The vendor knows their own seed**
Order placed	**Nothing attributable: commitments, nullifiers, and encrypted logs indistinguishable from any other Aztec activity**	**Nothing: an order's escrow is its own address, and an observer cannot tell it exists**	**Buyer and vendor: the full order, amount, and memo**
Order accepted	**One anonymous flag against an opaque id**	**same**	**The buyer sees the order progress**
Shipping update	**One anonymous fulfilment flag**	**same**	**The buyer sees shipped or delivered**
Order settled	**Nothing attributable**	**A terminal state on an opaque order id; nothing about the money**	**Vendor and owner receive private notes**
Dispute opened	**One anonymous flag, plus an opaque authentication commitment**	**same**	**The three participants converse off-chain**
Dispute ruled	**An anonymous outcome code on an opaque order id**	**same**	**The winning party collects from the order's escrow themselves**

Delivery addresses deserve a special word, because they are the most sensitive data in any real marketplace. They exist only inside a memo note encrypted to the vendor's inbox. They are never in public state, never in calldata, and never readable by the market operator.

Now consider an observer who *does* hold the market link: a customer, or anyone the link was forwarded to. They can read the storefront, browse listings, verify prices, and see vendor pseudonyms and their reputations. They still cannot see who bought anything, what any individual order contained, or what balance anyone holds. The link buys read access to the catalogue, not to the trade.

15. Honest limitations

A privacy system that hides its caveats should not be trusted. These are ours.

The link is the whole secret

Anyone who obtains a market link can read the catalogue. There is no revocation and no per-user access. Treat it as a shared password. The same is true of a seed: these are bearer instruments, and losing one is unrecoverable.

Volume is visible

Counters, note commitments, nullifiers, and the anonymous acceptance and dispute flags are public. An observer cannot tell what happened, but can tell roughly how much is happening, and when.

Timing correlates

An order placed at 14:02 and an acceptance at 14:03 on the same market are statistically related even though both are anonymous. Vendors who batch their responses blunt this; the protocol cannot remove it.

The rules are public

Fees, deposits, collateral, and timeouts are readable by anyone. This is a deliberate trade: the terms of the agreement should be checkable.

Off-chain content depends on Arweave

Listing content is encrypted, so Arweave learns nothing, but availability depends on it. An encrypted pointer is useless if the object cannot be fetched.

Moderators are trusted, within limits

A moderator can hide listings, suspend vendors, and rule on disputes. They cannot move money, cannot resolve an order twice, cannot touch an order the buyer already confirmed, and cannot deanonymise anyone. The limit on their power is structural rather than a promise of good behaviour, but within it, users are trusting the market's operator to rule fairly.

The operator is powerful but bounded

A market's owner can moderate listings, suspend vendors, rearrange the storefront, and change the rules for *future* orders. They cannot touch escrowed funds, which do not sit in their contract at all, cannot rewrite a placed order's terms, cannot read anyone's notes, and cannot swap the market's content without every link holder's client refusing to open it.

Shipping physical goods reveals a physical address

The delivery memo is encrypted to the vendor alone, but the vendor does learn where to send the parcel. Cryptography ends where the postal system begins.

16. Glossary

Access secret

The random number carried in a market link. Derives the keys that decrypt the market's content, its category tags, and its price blindings. It derives no key to any money: the marketplace contract has no encryption keys at all, and each order's escrow uses a secret minted for that order alone.

Authwit

Authorisation witness: a one-time, scoped permission an account signs so that another party may perform a precise action on its behalf inside one transaction.

Commitment
A hash that binds a value without revealing it. Used here for a listing's whole price table, for usernames, and for dispute authentication.

Price table
A listing's variants and shipping methods, each priced. One on-chain commitment binds all of it, and an order records the buyer's choice as a position in the lists rather than as a price, so the contract can recompute the total for itself.

Escrow keys
Per order, not per market. Each order's escrow derives its keys from a secret minted for that order and known only to its two parties, so a leak exposes one order rather than every order on the market. Spending stays gated by the escrow's own rules. The marketplace contract itself has no encryption keys, so holding a market link lets you decrypt nothing about it.

Fee juice
What Aztec transactions are paid in.

Note
An encrypted record representing private state: a balance fragment, an order, a receipt. Only its owner can read it; the chain stores only a commitment.

Nullifier
A one-way tag published when a note is spent or a one-time action is taken. The network rejects duplicates, which prevents double-spending and double-settlement without revealing what was spent.

Per-market account
A pseudonymous account derived from a user's seed and a specific market's link. The identity that appears on-chain within that market.

PXE
Private eXecution Environment: the wallet-side engine that holds keys, decrypts notes, runs private functions, and builds proofs.

Price commitment
A public hash binding a listing's hidden price, so that buyers can prove in-circuit that their payment matches it.

Sequencer
The network operator that orders transactions and executes their public parts. It verifies proofs of the private parts but cannot see inside them.

Universal wallet
The account that holds funds and fee credit and pays for transactions, including those performed on behalf of a per-market account.

Zero-knowledge proof

A short certificate that a computation was performed correctly, verifiable by anyone without seeing the computation's inputs.
